

SIMATS ENGINEERING

ASSESSMENT TOOL 3 — Flow Chart to Code Conversion

COURSE NAME: Data Structures

COURSE CODE: CSA03

COURSE OUTCOME COVERED

CO: Construct MST using shortest path algorithms and traverse the graph to model and analyse real-world problems. (Bloom Level - BL4, BL6)

Assessment Tool Weightage: 35%

QUESTIONS: Each question carries 10 marks **Duration:** 1 hour

Name: Koushik P **Reg No:** 192421178

Question 2

Convert a flowchart for traversing an array and displaying all its elements into a C program.

```
#include <stdio.h>
int main() {
    int arr[100], n, i;
    printf("Enter number of elements: ");
    scanf("%d", &n);
    printf("Enter %d elements: ", n);
    for (i = 0; i < n; i++)
        scanf("%d", &arr[i]);

    printf("Array elements are: ");
    for (i = 0; i < n; i++)
        printf("%d ", arr[i]);

    return 0;
}
```

Question 3

A flowchart accepts N elements into an array and finds the largest element. Convert the flowchart into C code.

```
#include <stdio.h>
int main() {
    int arr[100], n, i, largest;
    printf("Enter number of elements: ");
    scanf("%d", &n);
    printf("Enter %d elements: ", n);
    for (i = 0; i < n; i++)
        scanf("%d", &arr[i]);

    largest = arr[0];
    for (i = 1; i < n; i++)
        if (arr[i] > largest)
            largest = arr[i];

    printf("Largest element = %d\n", largest);
    return 0;
}
```

Question 4

Convert a flowchart for finding the smallest element in an array into a C program.

```
#include <stdio.h>
int main() {
    int arr[100], n, i, smallest;
    printf("Enter number of elements: ");
    scanf("%d", &n);
    printf("Enter %d elements: ", n);
    for (i = 0; i < n; i++)
        scanf("%d", &arr[i]);

    smallest = arr[0];
    for (i = 1; i < n; i++)
        if (arr[i] < smallest)
            smallest = arr[i];

    printf("Smallest element = %d\n", smallest);
    return 0;
}
```

Question 5

A flowchart performs linear search for a given key in an array. Convert it into C code.

```
#include <stdio.h>
int main() {
    int arr[100], n, i, key, found = -1;
    printf("Enter number of elements: ");
    scanf("%d", &n);
    printf("Enter %d elements: ", n);
    for (i = 0; i < n; i++)
        scanf("%d", &arr[i]);

    printf("Enter key to search: ");
    scanf("%d", &key);

    for (i = 0; i < n; i++) {
        if (arr[i] == key) {
            found = i;
            break;
        }
    }

    if (found != -1)
        printf("Element found at position %d\n", found + 1);
    else
        printf("Element not found\n");
    return 0;
}
```

Question 6

Convert a flowchart for binary search on a sorted array into a C program.

```
#include <stdio.h>
int main() {
    int arr[100], n, i, key, low, high, mid, found = -1;
    printf("Enter number of elements: ");
    scanf("%d", &n);
```

```

printf("Enter %d sorted elements: ", n);
for (i = 0; i < n; i++)
    scanf("%d", &arr[i]);

printf("Enter key to search: ");
scanf("%d", &key);

low = 0; high = n - 1;
while (low <= high) {
    mid = (low + high) / 2;
    if (arr[mid] == key) { found = mid; break; }
    else if (arr[mid] < key) low = mid + 1;
    else high = mid - 1;
}

if (found != -1)
    printf("Element found at position %d\n", found + 1);
else
    printf("Element not found\n");
return 0;
}

```

Question 7

A flowchart inserts an element at a specified position in an array by shifting existing elements. Convert it into C code.

```

#include <stdio.h>
int main() {
    int arr[100], n, i, pos, value;
    printf("Enter number of elements: ");
    scanf("%d", &n);
    printf("Enter %d elements: ", n);
    for (i = 0; i < n; i++)
        scanf("%d", &arr[i]);

    printf("Enter position to insert (1 to %d): ", n + 1);
    scanf("%d", &pos);
    printf("Enter value to insert: ");
    scanf("%d", &value);

    for (i = n; i >= pos; i--)
        arr[i] = arr[i - 1];
    arr[pos - 1] = value;
    n++;

    printf("Array after insertion: ");
    for (i = 0; i < n; i++)
        printf("%d ", arr[i]);
    return 0;
}

```

Question 8

A flowchart implements bubble sort by repeatedly comparing adjacent elements and swapping them. Convert the flowchart into C code.

```

#include <stdio.h>
int main() {
    int arr[100], n, i, j, temp;
    printf("Enter number of elements: ");

```

```

scanf("%d", &n);
printf("Enter %d elements: ", n);
for (i = 0; i < n; i++)
    scanf("%d", &arr[i]);

for (i = 0; i < n - 1; i++) {
    for (j = 0; j < n - i - 1; j++) {
        if (arr[j] > arr[j + 1]) {
            temp = arr[j];
            arr[j] = arr[j + 1];
            arr[j + 1] = temp;
        }
    }
}

printf("Sorted array: ");
for (i = 0; i < n; i++)
    printf("%d ", arr[i]);
return 0;
}

```

Question 9

Convert a flowchart for selection sort into a C program.

```

#include <stdio.h>
int main() {
    int arr[100], n, i, j, minIdx, temp;
    printf("Enter number of elements: ");
    scanf("%d", &n);
    printf("Enter %d elements: ", n);
    for (i = 0; i < n; i++)
        scanf("%d", &arr[i]);

    for (i = 0; i < n - 1; i++) {
        minIdx = i;
        for (j = i + 1; j < n; j++)
            if (arr[j] < arr[minIdx])
                minIdx = j;
        temp = arr[i]; arr[i] = arr[minIdx]; arr[minIdx] = temp;
    }

    printf("Sorted array: ");
    for (i = 0; i < n; i++)
        printf("%d ", arr[i]);
    return 0;
}

```

Question 10

A flowchart creates a singly linked list by inserting nodes at the end. Convert it into C code.

```

#include <stdio.h>
#include <stdlib.h>
struct Node {
    int data;
    struct Node* next;
};
struct Node* head = NULL;

void insertEnd(int value) {

```

```

    struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
    newNode->data = value;
    newNode->next = NULL;
    if (head == NULL) { head = newNode; return; }
    struct Node* temp = head;
    while (temp->next != NULL) temp = temp->next;
    temp->next = newNode;
}

void display() {
    struct Node* temp = head;
    while (temp != NULL) {
        printf("%d -> ", temp->data);
        temp = temp->next;
    }
    printf("NULL\n");
}

int main() {
    int n, value, i;
    printf("Enter number of nodes: ");
    scanf("%d", &n);
    for (i = 0; i < n; i++) {
        printf("Enter value: ");
        scanf("%d", &value);
        insertEnd(value);
    }
    display();
    return 0;
}

```

Question 11

Convert a flowchart for traversing a singly linked list and displaying each node into a C program.

```

#include <stdio.h>
#include <stdlib.h>
struct Node { int data; struct Node* next; };

void traverse(struct Node* head) {
    struct Node* temp = head;
    printf("Linked list elements: ");
    while (temp != NULL) {
        printf("%d ", temp->data);
        temp = temp->next;
    }
    printf("\n");
}

int main() {
    struct Node *head, *second, *third;
    head = (struct Node*)malloc(sizeof(struct Node));
    second = (struct Node*)malloc(sizeof(struct Node));
    third = (struct Node*)malloc(sizeof(struct Node));

    head->data = 10; head->next = second;
    second->data = 20; second->next = third;
    third->data = 30; third->next = NULL;

    traverse(head);
}

```

```
    return 0;
}
```

Question 12

Convert a flowchart for Depth-First Search (DFS) using a Stack into a C program.

```
#include <stdio.h>
#define MAX 100
int adj[MAX][MAX], visited[MAX], n;
int stack[MAX], top = -1;

void push(int v) { stack[++top] = v; }
int pop() { return stack[top--]; }
int isEmpty() { return top == -1; }

void DFS(int start) {
    push(start);
    while (!isEmpty()) {
        int v = pop();
        if (!visited[v]) {
            visited[v] = 1;
            printf("%d ", v);
            for (int i = n - 1; i >= 0; i--)
                if (adj[v][i] == 1 && !visited[i])
                    push(i);
        }
    }
}

int main() {
    int i, j, start;
    printf("Enter number of vertices: ");
    scanf("%d", &n);
    printf("Enter adjacency matrix:\n");
    for (i = 0; i < n; i++)
        for (j = 0; j < n; j++)
            scanf("%d", &adj[i][j]);

    printf("Enter starting vertex: ");
    scanf("%d", &start);
    printf("DFS traversal: ");
    DFS(start);
    return 0;
}
```

Question 13

Convert a flowchart for deleting the first node from a singly linked list into a C program.

```
#include <stdio.h>
#include <stdlib.h>
struct Node { int data; struct Node* next; };
struct Node* head = NULL;

void insertEnd(int value) {
    struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
    newNode->data = value; newNode->next = NULL;
    if (head == NULL) { head = newNode; return; }
    struct Node* temp = head;
    while (temp->next != NULL) temp = temp->next;
}
```

```

    temp->next = newNode;
}

void deleteFirst() {
    if (head == NULL) { printf("List is empty\n"); return; }
    struct Node* temp = head;
    head = head->next;
    printf("Deleted node with value %d\n", temp->data);
    free(temp);
}

void display() {
    struct Node* temp = head;
    while (temp != NULL) { printf("%d -> ", temp->data); temp = temp->next; }
    printf("NULL\n");
}

int main() {
    insertEnd(10); insertEnd(20); insertEnd(30);
    printf("Before deletion: "); display();
    deleteFirst();
    printf("After deletion: "); display();
    return 0;
}

```

Question 14

A flowchart implements Dijkstra's Algorithm. Convert it into C code.

```

#include <stdio.h>
#define MAX 100
#define INF 9999

int main() {
    int graph[MAX][MAX], n, i, j, src;
    int dist[MAX], visited[MAX];

    printf("Enter number of vertices: ");
    scanf("%d", &n);
    printf("Enter adjacency matrix (use %d for no edge):\n", INF);
    for (i = 0; i < n; i++)
        for (j = 0; j < n; j++)
            scanf("%d", &graph[i][j]);

    printf("Enter source vertex:");
    scanf("%d", &src);

    for (i = 0; i < n; i++) { dist[i] = INF; visited[i] = 0; }
    dist[src] = 0;

    for (int count = 0; count < n - 1; count++) {
        int u = -1, minDist = INF;
        for (i = 0; i < n; i++)
            if (!visited[i] && dist[i] < minDist) { minDist = dist[i]; u = i; }
        if (u == -1) break;
        visited[u] = 1;
        for (j = 0; j < n; j++)
            if (!visited[j] && graph[u][j] != INF && dist[u] + graph[u][j] < dist[j])
                dist[j] = dist[u] + graph[u][j];
    }
}

```

```

    printf("Vertex\tDistance from Source\n");
    for (i = 0; i < n; i++)
        printf("%d\t%d\n", i, dist[i]);
    return 0;
}

```

Question 15

Convert a flowchart for checking stack overflow and underflow conditions into a C program.

```

#include <stdio.h>
#define MAX 5
int stack[MAX], top = -1;

void push(int value) {
    if (top == MAX - 1) { printf("Stack Overflow\n"); return; }
    stack[++top] = value;
    printf("%d pushed to stack\n", value);
}

void pop() {
    if (top == -1) { printf("Stack Underflow\n"); return; }
    printf("%d popped from stack\n", stack[top--]);
}

int main() {
    push(10); push(20); push(30); push(40); push(50);
    push(60); // triggers overflow (stack size = 5)
    pop(); pop(); pop(); pop(); pop();
    pop(); // triggers underflow
    return 0;
}

```

Question 16

A flowchart implements a queue using an array with ENQUEUE and DEQUEUE operations. Convert it into C code.

```

#include <stdio.h>
#define MAX 5
int queue[MAX], front = -1, rear = -1;

void enqueue(int value) {
    if (rear == MAX - 1) { printf("Queue Overflow\n"); return; }
    if (front == -1) front = 0;
    queue[++rear] = value;
    printf("%d enqueued\n", value);
}

void dequeue() {
    if (front == -1 || front > rear) { printf("Queue Underflow\n"); return; }
    printf("%d dequeued\n", queue[front++]);
}

int main() {
    enqueue(10); enqueue(20); enqueue(30);
    dequeue(); dequeue();
    enqueue(40);
    dequeue(); dequeue();
    dequeue(); // triggers underflow
}

```

```

    return 0;
}

```

Question 17

Convert a flowchart for implementing Kruskal's Algorithm into a C program.

```

#include <stdio.h>
#define MAX 100
int parent[MAX];

int find(int i) {
    while (parent[i] != i) i = parent[i];
    return i;
}

void unionSet(int i, int j) {
    int a = find(i), b = find(j);
    parent[a] = b;
}

struct Edge { int u, v, w; };

int main() {
    int n, e, i, j;
    struct Edge edges[MAX], temp;

    printf("Enter number of vertices: ");
    scanf("%d", &n);
    printf("Enter number of edges: ");
    scanf("%d", &e);
    printf("Enter edges (u v weight):\n");
    for (i = 0; i < e; i++)
        scanf("%d %d %d", &edges[i].u, &edges[i].v, &edges[i].w);

    for (i = 0; i < e - 1; i++)
        for (j = 0; j < e - i - 1; j++)
            if (edges[j].w > edges[j + 1].w) {
                temp = edges[j]; edges[j] = edges[j + 1]; edges[j + 1] = temp;
            }

    for (i = 0; i < n; i++) parent[i] = i;

    int cost = 0, count = 0;
    printf("Edges in MST:\n");
    for (i = 0; i < e && count < n - 1; i++) {
        int u = edges[i].u, v = edges[i].v;
        if (find(u) != find(v)) {
            printf("%d - %d : %d\n", u, v, edges[i].w);
            cost += edges[i].w;
            unionSet(u, v);
            count++;
        }
    }
    printf("Total MST cost = %d\n", cost);
    return 0;
}

```

Question 18

A flowchart performs Prim's Algorithm. Convert the flowchart into a recursive C program.

```

#include <stdio.h>
#define MAX 100
#define INF 9999
int graph[MAX][MAX], selected[MAX], n;

void primRecursive(int edgeCount) {
    if (edgeCount == n - 1) return;    // base case: MST complete

    int minEdge = INF, x = -1, y = -1, i, j;
    for (i = 0; i < n; i++) {
        if (selected[i]) {
            for (j = 0; j < n; j++) {
                if (!selected[j] && graph[i][j] && graph[i][j] < minEdge) {
                    minEdge = graph[i][j]; x = i; y = j;
                }
            }
        }
    }

    printf("%d - %d : %d\n", x, y, minEdge);
    selected[y] = 1;
    primRecursive(edgeCount + 1);    // recursive call
}

int main() {
    int i, j;
    printf("Enter number of vertices: ");
    scanf("%d", &n);
    printf("Enter adjacency matrix (0 for no edge):\n");
    for (i = 0; i < n; i++)
        for (j = 0; j < n; j++)
            scanf("%d", &graph[i][j]);

    for (i = 0; i < n; i++) selected[i] = 0;
    selected[0] = 1;

    printf("Edges in MST:\n");
    primRecursive(0);
    return 0;
}

```

Question 19

Convert a flowchart for Breadth-First Search (BFS) using a queue into a C program.

```

#include <stdio.h>
#define MAX 100
int adj[MAX][MAX], visited[MAX], n;
int queue[MAX], front = -1, rear = -1;

void enqueue(int v) { queue[++rear] = v; if (front == -1) front = 0; }
int dequeue() { return queue[front++]; }
int isEmpty() { return front == -1 || front > rear; }

void BFS(int start) {
    enqueue(start);
    visited[start] = 1;
    while (!isEmpty()) {
        int v = dequeue();
        printf("%d ", v);
        for (int i = 0; i < n; i++) {

```

```

        if (adj[v][i] == 1 && !visited[i]) {
            visited[i] = 1;
            enqueue(i);
        }
    }
}

int main() {
    int i, j, start;
    printf("Enter number of vertices: ");
    scanf("%d", &n);
    printf("Enter adjacency matrix:\n");
    for (i = 0; i < n; i++)
        for (j = 0; j < n; j++)
            scanf("%d", &adj[i][j]);

    printf("Enter starting vertex: ");
    scanf("%d", &start);
    printf("BFS traversal: ");
    BFS(start);
    return 0;
}

```

Question 20

Application-based: A flowchart accepts a set of integers, stores them in a data structure, searches for a specified element, and displays its position. Identify the appropriate data structure and convert the complete flowchart into a C program.

```

#include <stdio.h>
int main() {
    int arr[100], n, i, key, found = -1;
    printf("Enter number of elements: ");
    scanf("%d", &n);
    printf("Enter %d integers: ", n);
    for (i = 0; i < n; i++)
        scanf("%d", &arr[i]);

    printf("Enter element to search: ");
    scanf("%d", &key);

    for (i = 0; i < n; i++) {
        if (arr[i] == key) { found = i; break; }
    }

    if (found != -1)
        printf("Element %d found at position %d\n", key, found + 1);
    else
        printf("Element %d not found in the array\n", key);
    return 0;
}

```

Data structure chosen: an Array. Since the task only needs indexed storage of a fixed set of integers and a simple sequential search to report position, a one-dimensional array is the simplest and most appropriate structure — no dynamic insertion/deletion or ordering is required.